

模型能力

思考模式

[Copy page](#)

GLM 提供多种思考模式，覆盖从常规对话到工具调用与编码智能体的不同需求。下文将分别说明各模式的启用方式、关键注意事项与示例用法。

默认思考行为

GLM-5.1 GLM-5 GLM-4.7 系列默认开启 Thinking，这一点不同于 GLM-4.6 的默认“混合 thinking（自动开启）”。

如果你想关闭 thinking，请使用：

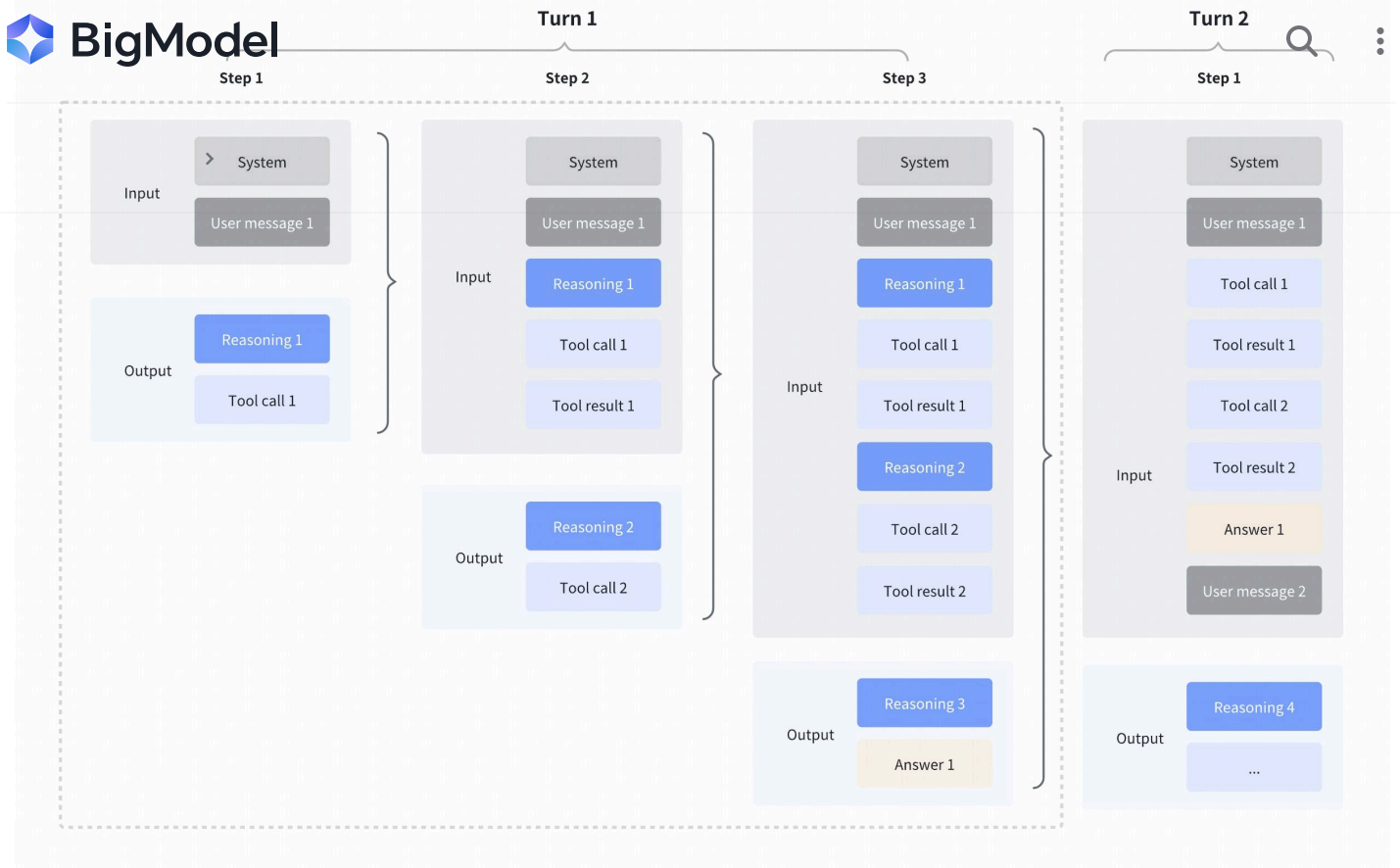
```
"thinking": {  
  "type": "disabled"  
}
```

交错式思考（Interleaved thinking）

我们默认支持交错式思考（这一点从 GLM 4.5 开始就支持），使 GLM 可以在工具调用之间、以及收到工具结果之后继续思考。这让模型能够进行更复杂的分步推理：在决定下一步行动前先解读每次工具输出，把多次工具调用与推理步骤串联起来，并根据中间结果做更细粒度的决策。

ⓘ 注意：当你在使用“交错思考 + 工具”时，必须显式保留 Reasoning content，并在返回工具结果时一并返回

交错思考的详细流程示意如下：

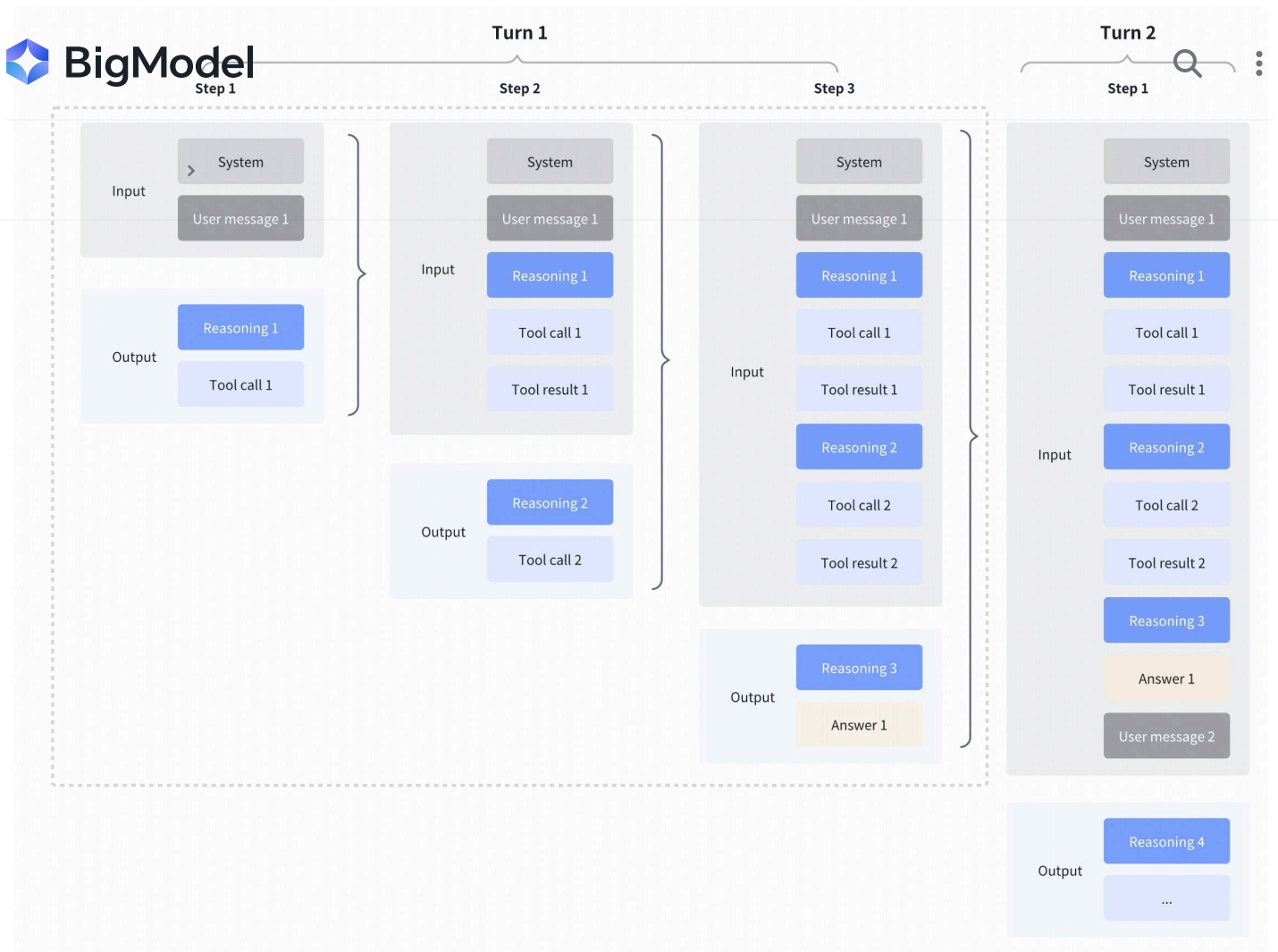


保留式思考 (Preserved thinking)

我们在编码场景中引入了一项新能力：模型可以在上下文中保留来自先前 assistant 回合的 reasoning content。这有助于保持推理连续性与对话完整性、提升模型表现，并提高缓存命中率，在真实任务中节省更多 tokens。

- ✓ 该能力在 Coding Plan 端点默认开启、标准 API 端点默认关闭。如果你想在你的产品中开启保留式思考（该能力主要推荐 Coding / Agent 场景使用），你可以通过「`"clear_thinking": False`」在 API 端点中开启，并需要将完整、未修改的 reasoning content 传回 API。所有连续的 reasoning content 必须与模型在原始请求期间生成的序列完全一致，不要重新排序或修改这些 content，否则会降低效果并影响缓存命中。

保留式思考的详细流程示意如下：



轮级思考

「轮级思考（Turn-level Thinking）」是一种**按轮控制推理计算**的能力：在同一调用会话中，每一轮请求都可以独立选择开启/关闭思考。这是 GLM-4.7 新引入的能力，具备以下优势：

- **更灵活的成本/时延控制**：对“问个事实/改个措辞”等轻量轮次可关闭思考，追求快速响应；对“复杂规划/多约束推理/代码调试”等重任务轮次可开启思考，提升正确率与稳定性。
- **更顺滑的多轮体验**：思考开关在会话内可随时切换，模型能在不同轮次间保持对话连贯与输出风格一致，让用户感觉“聪明时更聪明、简单时更快”。
- **更适合 Agent / 工具调用场景**：在需要快速执行的工具轮次可降低推理开销，在需要综合工具结果做决策的轮次再开启深度思考，实现效率与质量的动态平衡。

使用示例

该机制同时适用于 Interleaved Thinking 和 Preserved Thinking，无需手动区分。



请记得返回历史的 `reasoning_content`，以保持推理连贯性。

>



```
import json
from openai import OpenAI

client = OpenAI(
    api_key="YOUR_API_KEY",
    base_url="https://api.z.ai/api/paas/v4/",
)

tools = [{"type": "function", "function": {
    "name": "get_weather",
    "description": "Get weather information",
    "parameters": {"type": "object", "properties": {"city": {"type": "string"}},
}}]

messages = [
    {"role": "system", "content": "You are an assistant"},
    {"role": "user", "content": "What's the weather like in Beijing?"},
]

# Round 1: the model reasons and then calls a tool
response = client.chat.completions.create(model="glm-4.7", messages=messages, tools=tools,
    thinking={
        "type": "enabled",
        "clear_thinking": False # False for Preserved Thinking
    })
reasoning, content, tool_calls = "", "", []
for chunk in response:
    delta = chunk.choices[0].delta
    if hasattr(delta, "reasoning_content") and delta.reasoning_content:
        reasoning += delta.reasoning_content
    if hasattr(delta, "content") and delta.content:
        content += delta.content
    if hasattr(delta, "tool_calls") and delta.tool_calls:
        for tc in delta.tool_calls:
            if tc.index >= len(tool_calls):
                tool_calls.append({"id": tc.id, "function": {"name": "", "arguments": ""}})
            if tc.function.name:
                tool_calls[tc.index]["function"]["name"] = tc.function.name
```



```
tool_calls[tc.index]["function"]["arguments"] += tc.function.arguments
```

```
# Key: return reasoning_content to keep the reasoning coherent
messages.append({"role": "assistant", "content": content, "reasoning_content": reasoning_content,
                 "tool_calls": [{"id": tc["id"], "type": "function", "function": {"name": "get_weather", "arguments": {"location": "London", "date": "2024-07-20"}}}]}
messages.append({"role": "tool", "tool_call_id": tool_calls[0]["id"],
                 "content": json.dumps({"weather": "Sunny", "temp": "25°C"})})

# Round 2: the model continues reasoning based on the tool result and responds
response = client.chat.completions.create(model="glm-4.7", messages=messages, tool_choice="auto")

reasoning_content = ""
content = ""

for chunk in response:
    delta = chunk.choices[0].delta
    if hasattr(delta, "reasoning_content") and delta.reasoning_content:
        reasoning_content += delta.reasoning_content
    if hasattr(delta, "content") and delta.content:
        content += delta.content

print(f"Reasoning: {reasoning_content}\nReply: {content}")
```